In both the array and `if` cases, if the logic gets complicated, separating out to another function or possibly another component makes it clearer. You can nest functions in JavaScript, so you could do something like this:

```
const Conditional = (props) => {

 function spanItem {
  return (props.x === true) ? <span>True</span> : <span>False</span>
 }

 return <div>{spanItem()}</div>
}
```

There's one other React quirk that you need to know about when you are generating lists of items. If you look at our creation of the `Seat` component, you'll see that the call looks like this:

```
<Seat key={seatNumber} seatNumber={seatNumber} />
```

The `seatNumber` goes into the `props` for the component, but the `key` is new. The `key` is an internal property used by React to enable the React run time to differentiate between elements in a list and render only the ones that have changed. The exact value of the key doesn't much matter for our purposes right now. The only thing that matters is that each element's key is unique. If you render a list without a key, React will place a warning in the browser console at run time, so that's something to look out for.

So far the JSX tags we've written have all been self-closing, ending in `/>`. But you can have beginning and ending tags with arbitrary expressions inside the tags. These expressions can be a string, like `<Tag>Fred</Tag>`; one or more JSX elements, like `<Header><Nav/><Nav/></Header>`; or an arbitrary expression, like `<DisplayUser>{user}</DisplayUser>`. In all these cases, the value inside the tags is available as `props.children`, and the parent receiving them is responsible for doing whatever makes sense with the `props.children` value so that the child elements are used.

TypeScript and Child Props